

```
❶ fig, ax = plt.subplots()
   ax.plot(squares)

plt.show()
```

We first import the `pyplot` module using the alias `plt` so we don't have to type `pyplot` repeatedly. (You'll see this convention often in online examples, so we'll use it here.) The `pyplot` module contains a number of functions that help generate charts and plots.

We create a list called `squares` to hold the data that we'll plot. Then we follow another common Matplotlib convention by calling the `subplots()` function ❶. This function can generate one or more plots in the same figure. The variable `fig` represents the entire *figure*, which is the collection of plots that are generated. The variable `ax` represents a single plot in the figure; this is the variable we'll use most of the time when defining and customizing a single plot.

We then use the `plot()` method, which tries to plot the data it's given in a meaningful way. The function `plt.show()` opens Matplotlib's viewer and displays the plot, as shown in Figure 15-1. The viewer allows you to zoom and navigate the plot, and you can save any plot images you like by clicking the disk icon.

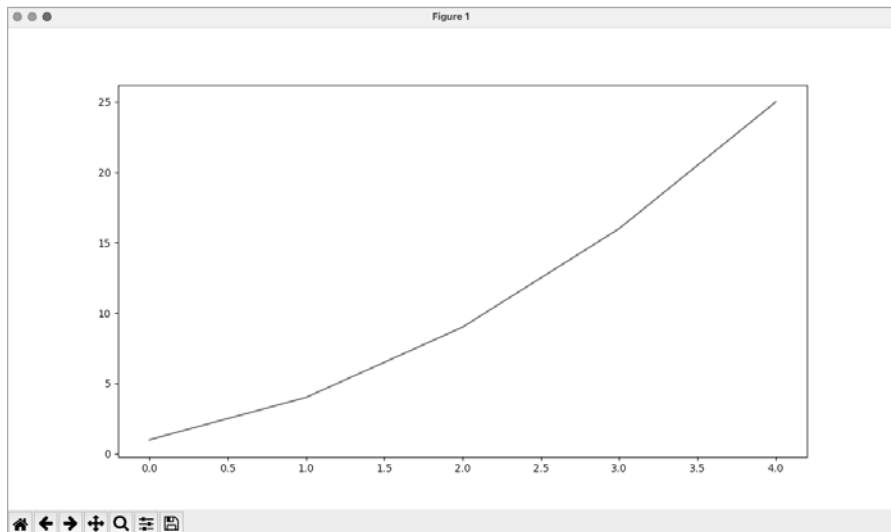


Figure 15-1: One of the simplest plots you can make in Matplotlib

Changing the Label Type and Line Thickness

Although the plot in Figure 15-1 shows that the numbers are increasing, the label type is too small and the line is a little thin to read easily. Fortunately, Matplotlib allows you to adjust every feature of a visualization.